Mini-aula: etcd e ZooKeeper

O essencial para a apresentação. Os números entre parênteses são os que medimos nos experimentos.

1. O que são e para que servem

O etcd e o ZooKeeper são **serviços de coordenação**: guardam pouca informação, mas informação crítica, como a configuração de um sistema, quem é o líder de um serviço ou quem está usando um recurso. O Kubernetes, por exemplo, guarda todo o estado do cluster no etcd; o Kafka (antes do KRaft) e o HBase usam o ZooKeeper.

O objetivo dos dois é o mesmo: **não ter ponto único de falha e nunca ter duas versões do mesmo dado**. Para isso, cada um roda em 3 servidores, os **nós**, e cada nó guarda uma cópia inteira dos dados. As cópias ficam iguais porque todas aplicam os mesmos comandos, na mesma ordem; isso se chama **máquina de estados replicada**. A ordem dos comandos é decidida por um algoritmo de **consenso**: o **Raft**, no etcd, e o **ZAB**, no ZooKeeper.

Os programas que usam esses serviços são os **clientes**. Eles ficam de fora: não votam e não guardam cópia; só pedem "grava isto" ou "lê aquilo" a qualquer um dos nós.

2. Quem manda: líder, seguidores e mandato

Em vez de os três decidirem tudo juntos o tempo todo, eles elegem um **líder**. Só o líder decide a ordem das escritas; os outros são **seguidores** e copiam o que ele manda.

A liderança tem um mandato numerado. No etcd ele se chama **termo**; no ZooKeeper, **época**. A cada eleição, o número sobe 1. Toda mensagem leva o número do mandato, e mensagem de mandato antigo é recusada: assim, um líder antigo que volta não consegue dar ordens.

Tudo depende da **maioria**: com 3 nós, a maioria é 2. É preciso maioria para eleger e para confirmar uma escrita. Como duas maiorias sempre têm um nó em comum, nunca existem duas decisões conflitantes. Com 3 nós, o sistema aguenta a falha de 1.

3. Como os nós se conectam

Nos nossos laboratórios, cada sistema tem duas redes: a **interna**, só para os nós conversarem entre si (o consenso), e a **de clientes**, por onde chegam os pedidos.

No **etcd**, o líder manda pela rede interna um aviso de "estou vivo", o **heartbeat**, a cada 100 ms. Os seguidores só percebem que o líder sumiu pela **falta** desses avisos.

No **ZooKeeper**, cada seguidor mantém uma **conexão TCP permanente** com o líder, como uma ligação telefônica aberta. Se o processo do líder morre, as conexões dele se fecham, e os seguidores sabem **na hora**. Se a conexão só fica muda, sem fechar, eles esperam um tempo limite: 4 s na configuração da imagem que usamos.

4. Como escolhem o líder

etcd (Raft). Cada seguidor tem um cronômetro sorteado entre 1 e 2 s, zerado a cada heartbeat. Se o cronômetro estoura, o seguidor vira **candidato**: soma 1 ao termo, vota em si e pede votos. Cada nó vota uma única vez por termo, e só num candidato com histórico pelo menos tão atualizado quanto o seu. Com 2 votos, ele vira líder. O sorteio existe para que dois seguidores raramente se candidatem juntos e dividam os votos; por isso o vencedor é imprevisível, ao contrário do algoritmo do Valentão (Bully), em que ganha sempre o maior identificador.

ZooKeeper (ZAB). Quando os servidores percebem que não há líder, cada um vota em quem tem o **histórico mais novo** (o maior zxid, o número da última transação); em caso de empate, no de maior identificador. Eles trocam votos até a maioria concordar. O novo líder abre uma **época** nova, e o contador de transações recomeça do zero.

Nos dois casos, o novo líder tem obrigatoriamente todas as escritas já confirmadas.

5. Como funciona uma escrita

O caminho é o mesmo nos dois sistemas:

1. O cliente manda a escrita para qualquer nó; se ele não for o líder, **encaminha ao líder**.
2. O líder anota a escrita no seu registro, o **log**, e a envia aos seguidores.
3. Cada seguidor anota e responde "anotei".

4. Quando a **maioria** anotou (o líder e mais um), a escrita está **confirmada**.
5. Cada nó aplica a escrita na sua cópia quando fica sabendo da confirmação; o nó que recebeu o pedido aplica e responde OK ao cliente.

Sem maioria (2 nós fora do ar), nenhuma escrita é confirmada: o cliente recebe erro por tempo esgotado.

6. Como funciona uma leitura

etcd. A leitura padrão é **linearizável**: sempre devolve a última escrita confirmada, não importa em qual nó. Para isso, o nó pergunta ao líder qual é a posição confirmada; o líder confirma com a maioria que ainda é o líder; e o nó responde quando tiver chegado a essa posição (1,82 ms). Existe também a leitura **serializável**, opcional, que responde com a cópia local sem perguntar a ninguém: é mais rápida (0,65 ms), mas pode estar desatualizada.

ZooKeeper. É o contrário: a leitura padrão é **local** (1,03 ms) e pode estar atrasada. Para garantir o valor mais recente, o cliente chama **sync** antes de ler (3,25 ms).

Em resumo: o etcd é seguro por padrão, e você pede para relaxar; o ZooKeeper é rápido por padrão, e você pede para garantir.

7. Líder derrubado × líder desconectado

Derrubado (o processo morre, `docker kill`). No etcd, os seguidores só percebem pela falta de heartbeat: esperam o tempo sorteado e elegem outro (**1,46 s**). No ZooKeeper, a conexão fecha e eles percebem na hora (**0,38 s**). Com 2 de 3 nós, tudo continua funcionando. Se o etcd for desligado de forma educada (`docker stop`), o líder passa o cargo antes de sair (**9 ms**).

Desconectado (o líder continua vivo, mas fora da rede interna: uma **partição**). O lado com 2 nós para de ouvir o líder e elege outro, com mandato maior: **0,94 s** no etcd e **3,6 s** no ZooKeeper, que aqui é mais lento porque nada fecha e ele espera o limite de 4 s. E o líder isolado?

- No etcd, ele percebe que não ouve a maioria e **desiste de ser líder** ("no leader"). Recusa escritas e leituras linearizáveis; só a leitura serializável responde, com o valor antigo.
- No ZooKeeper, ele ainda serve leitura local antiga por alguns segundos (cerca de 3,6 s no E9-E4), até perceber que perdeu o líder, e então **para de**

atender completamente; o cliente que conhece os outros servidores troca de servidor sozinho.

Os dois escolhem **consistência em vez de disponibilidade** (teorema CAP): o lado minoritário não confirma nenhuma escrita (no máximo, serve uma leitura antiga), então nunca existem duas versões confirmadas do dado.

Quando o nó volta. Ele recebe uma mensagem com mandato maior que o seu e vira **seguidor**. O líder manda só o que ele perdeu: no etcd, as entradas que faltam, ou uma "foto" inteira dos dados, o **snapshot**, se o histórico antigo já foi apagado; no ZooKeeper, a sincronização **DIFF**. O que ele tentou gravar enquanto estava isolado nunca foi confirmado e simplesmente não existe.

8. Cola rápida

	etcd (Raft)	**ZooKeeper (ZAB)**
Mandato	termo	época (parte alta do zxid)
Percebe a queda do líder	falta de heartbeat (1 a 2 s)	conexão fechada (na hora) ou 4 s sem resposta
Quem vira líder	quem estoura o tempo primeiro e ganha a maioria	quem tem o histórico mais novo
Leitura padrão	linearizável (garantida)	local (pode estar atrasada)
Eleição após queda	1,46 s	0,38 s
Eleição na partição	0,94 s	3,6 s
Líder isolado	desiste e recusa escrita (a leitura serializável devolve o valor antigo)	serve leitura antiga por alguns segundos, depois para de atender
Nó que volta	vira seguidor; recebe o log ou um snapshot	vira seguidor; recebe um DIFF